Parallel Computing

Chapter 1: Why Parallelism? Why Efficiency?

Notes by Mihir Rao

The course has two interleaved aims: writing programs that run efficiently on modern parallel hardware, and understanding the hardware well enough to know why some programs scale and others do not. This first lecture is mostly motivation. We give a working definition of a parallel computer, introduce speedup, and explain why every modern processor — from a phone SoC to a data-center GPU — is parallel.

1.1 What Is a Parallel Computer?

> **Definition** [parallel computer]. A parallel computer is a collection of processing elements that cooperate to solve problems quickly.

Two words in this definition do real work. *Cooperate* implies communication and synchronisation between processing elements: they share data and coordinate progress. *Quickly* is the reason we bother in the first place — we are using more than one processor because we want the answer sooner, or because a single processor cannot keep up with the problem at all.

Two intertwined goals. *Throughout the course we care simultaneously about performance (getting the answer fast) and efficiency (using the hardware's resources well). They are related but not the same: a program can run faster on a parallel machine and still be terrible at using its execution units.*

Speedup

The first quantitative tool is speedup.

> **Definition** [speedup]. For a fixed problem, the speedup of a parallel implementation using P processors is

$$S(P) = \frac{T_1}{T_P},$$

where T_1 is the execution time on one processor and T_P is the execution time on P processors.

Ideally $S(P) = P$ (LINEAR SPEEDUP). In practice $S(P) < P$, often by a wide margin, for reasons we will study repeatedly:

- COMMUNICATION. Processors must exchange data. Moving data eventually dominates computation, especially as P grows.
- LOAD IMBALANCE. If some processors finish early and sit idle while others continue, the parallel time is set by the slowest, not the average.
- SYNCHRONISATION AND DEPENDENCIES. Some operations must wait for others to finish. The longest chain of dependent work bounds the achievable speedup regardless of P.

EXAMPLE [THREE CLASSROOM DEMOS]. In a small partial-sum demo with a handful of "processors" (students), communication cost was the binding constraint: putting students closer together or letting them shout improved speedup. With four "processors" the binding constraint shifted to load imbalance — some students ran out of work and went idle while others were still summing. With many "processors" communication once again dominated, and adding more people made the program SLOWER, not faster.

1.2 Three Course Themes

The class is organised around three themes, each a perspective on the same artefact.

Theme 1 — Designing parallel programs that scale

PARALLEL THINKING consists of three concrete tasks:

1. DECOMPOSING the work into pieces that can safely be performed in parallel.
2. ASSIGNING those pieces to processors.
3. MANAGING communication and synchronisation so that they do not negate the speedup we hoped to win.

We will study abstractions and mechanisms — ISPC, pthreads, OpenMP, CUDA, MPI — that let us express these decisions cleanly in code.

Theme 2 — How parallel hardware actually works

A parallel program lives on top of an implementation: caches, memory hierarchies, vector units, multiple cores, multiple sockets, GPUs, accelerators. Performance characteristics of those implementations and the tradeoffs behind them (performance vs. convenience vs. cost in silicon area, power, and chip complexity) are an explicit subject of the course, not a hidden detail.

> **Why hardware.** *Recall the demos: changing how processors communicated changed the speedup. Caring about performance forces us to care about the hardware.*

Theme 3 — Thinking about efficiency

Fast is not the same as efficient. A program that runs $2\times$ faster on a ten-processor machine has captured only 20% of the available parallelism; the other eight processors are wasted. From the programmer's side, efficiency means making use of the machine's capabilities; from the hardware designer's side, it means choosing capabilities that pay off in performance per unit silicon area or per watt.

1.3 Why Parallelism Now?

For decades, single-threaded CPU performance roughly doubled every 18 months. The rational software response was to wait: code that was too slow this year would be fast enough next year. Two engines drove that improvement:

1. exploiting instruction-level parallelism (ILP) via superscalar, out-of-order execution; and
2. increasing the CPU clock frequency.

Roughly fifteen years ago, both engines stalled. Understanding why is the reason this course exists.

Programs as instruction streams

From a processor's perspective, a program is a list of instructions. The small C program

```
int main(int argc, char** argv) {
    int x = 1;
    for (int i = 0; i < 10; i++) {
        x = x + x;
    }
    printf("%d\n", x);
    return 0;
}
```

compiles into a sequence of machine instructions of the form

```
movl  $1, -20(%rbp)
movl  $0, -24(%rbp)
cmpl  $10, -24(%rbp)
jge   end
movl  -20(%rbp), %eax
addl  -20(%rbp), %eax
...
```

A processor executes these one at a time: FETCH AND DECODE the next instruction, READ its inputs from registers, EXECUTE the operation in an arithmetic logic unit (ALU), WRITE BACK the result. On the simplest in-order processor each instruction takes one clock cycle, and the program runs in (number of instructions) $\times$ (cycle time) seconds.

Instruction-level parallelism

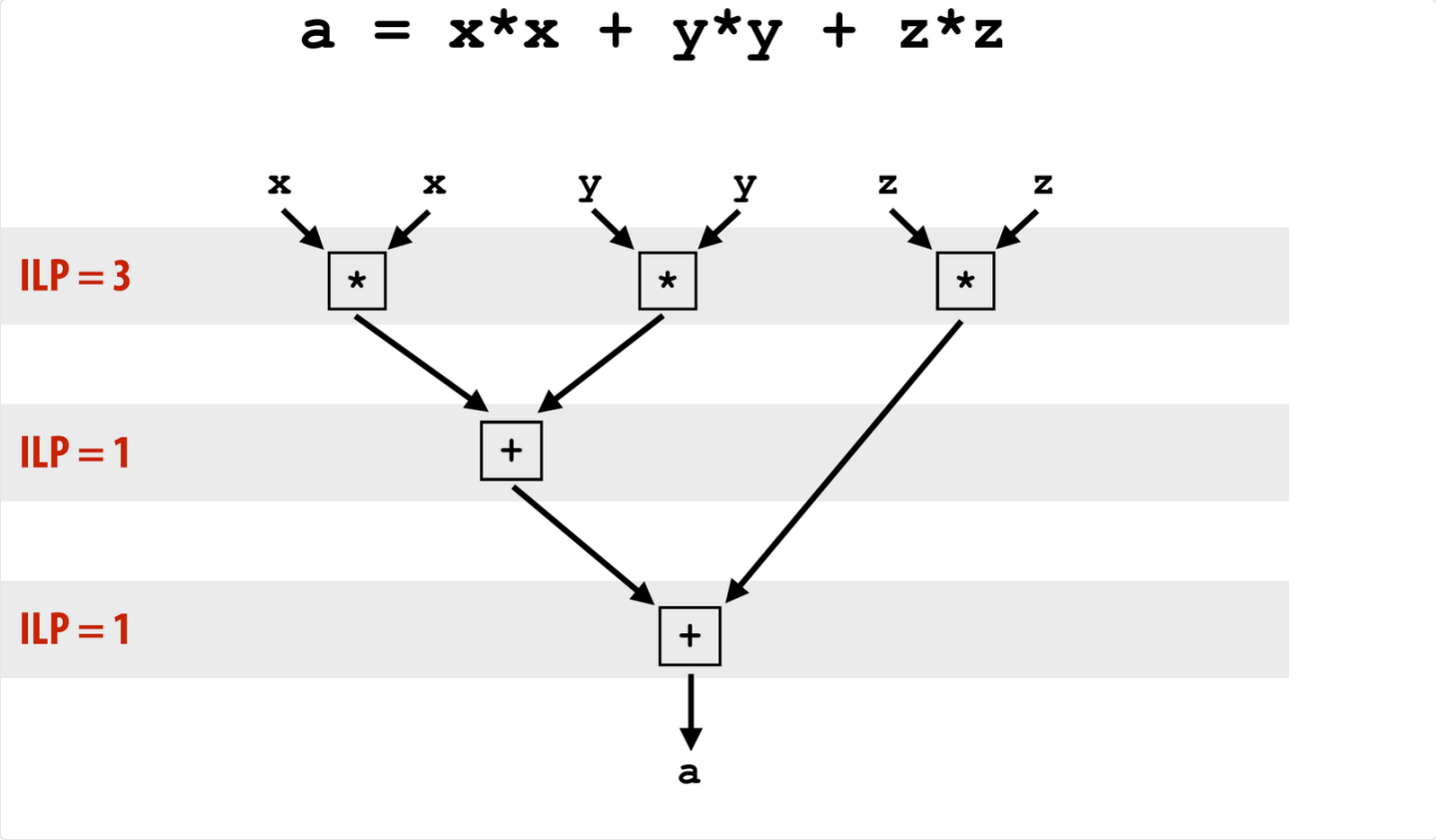
Consider the trivial computation

$$a = x^2 + y^2 + z^2,$$

which compiles to (with $R_0 = x$, $R_1 = y$, $R_2 = z$):

```
1   mul R0, R0, R0
2   mul R1, R1, R1
3   mul R2, R2, R2
4   add R0, R0, R1
5   add R3, R0, R2
```

A naïve in-order processor takes five cycles. But instructions 1, 2, 3 are mutually independent — disjoint inputs, disjoint outputs — so a processor with two ALUs can issue any two of them in the same cycle, and one with three can issue all three at once. Instruction 4 depends on 1 and 2, and instruction 5 depends on 4. The dependency graph has critical-path length 3, and three cycles is the best we can do regardless of how many ALUs we have.



Dependency graph for $a = x^2 + y^2 + z^2$. Critical-path length three — the minimum cycle count regardless of issue width.

> **Definition** [instruction-level parallelism]. The **instruction-level parallelism** (ILP) of a program at a given step is the number of instructions that can be issued simultaneously without violating dependencies. A **superscalar** processor finds independent instructions in the dynamic instruction stream and issues several of them per cycle on multiple execution units.

Empirically (Johnson, 1991), most of the ILP in real programs is captured by a processor that can issue about four instructions per clock; building wider machines yields rapidly diminishing returns. By the late 2000s designers had effectively saturated ILP: shipping a wider issue width no longer translated into proportionate speedup on real workloads.

The power wall

The other engine, clock-frequency scaling, hit a separate wall. Dynamic power dissipation in CMOS logic scales as

$$P_{\text{dyn}} \propto C \cdot V^2 \cdot f,$$

where C is the switched capacitive load, V is the core voltage, and f is the clock frequency. The maximum stable f depends on V, so pushing the clock requires raising voltage, and power grows roughly as f^3 in the relevant regime. Static (leakage) power adds a baseline that does not vanish when transistors are idle.

The practical consequence is a thermal design budget. Apple's M1 laptop chip sits around 13 W; an Intel Core i9 desktop CPU around 95 W; an NVIDIA RTX 4090 GPU around 450 W; a top-end supercomputer node measured in megawatts. Within any of these envelopes, you cannot simply crank the clock to recover speedups that ILP no longer delivers.

> **The two-wall summary.** *Single-thread performance growth slowed because (1) ILP scaling tapped out and (2) frequency scaling was capped by power. Architects responded by spending the available transistors on* MORE *execution units running in parallel rather than on a single faster core. The free lunch ended.*

1.4 The Parallel Hardware Landscape

Once the design budget is spent on parallelism rather than on faster scalar cores, the hardware landscape diversifies along several axes.

Multi-core CPUs

A multi-core CPU is essentially several scalar cores on the same die, each with its own fetch/decode and execution units, sharing a memory hierarchy. Intel's 10th-generation Core i9 has 10 cores; AMD's Ryzen Threadripper 3990X has 64 cores arranged as eight chiplets of eight cores each. Programs that decompose work across cores can in principle achieve nearly P-fold speedups, subject to the same communication / load-balance / dependency constraints we saw in the demos.

SIMD and hyper-threading

Inside a single core, two more layers of parallelism are exposed. SIMD (Single Instruction, Multiple Data, e.g. Intel AVX) instructions operate on a vector of elements per cycle. HYPER-THREADING runs two instruction streams in the same physical core, letting the core hide latency by switching between them. Combining all three layers on a quad-core CPU can yield roughly 32–$40\times$ speedup over a single-threaded baseline compiled with `-O3` — far more than the four-fold gain from cores alone.

GPUs and accelerators

A modern GPU is a different point in the design space: thousands of small, simple execution units rather than a few wide, complex ones. The NVIDIA RTX 4090 contains roughly 76 billion transistors and 18,432 FP32 multipliers organised as 144 streaming multiprocessors. Data-center supercomputers (Frontier at Oak Ridge: 9,472 AMD CPUs and 37,888 Radeon GPUs, 21 MW total power) are large-scale aggregations of these parts.

Specialisation

Efficiency, not just parallelism, is the second motivating force. The same power wall that makes parallelism necessary makes SPECIALISATION attractive: it is more energy-efficient to perform a fixed task on hardware designed exactly for it than to emulate that task on a general-purpose core. A modern phone SoC contains a CPU, a GPU, and a pile of fixed-function accelerators (image signal processor, video codec, neural-network engine). Data-center accelerators (Google TPU, GraphCore IPU, Cerebras Wafer-Scale Engine) push the same logic up to a different scale.

> **WHERE THIS COURSE GOES.** *Achieving efficient processing almost always reduces to accessing data efficiently. Memory hierarchy and locality will be a recurring subject: even on a heavily parallel machine, the cost of moving data between caches, DRAM, and other processors is what determines whether a program scales.*

1.5 Memory and the Cost of Data Movement

Achieving efficient processing almost always reduces to accessing data efficiently. Before we can study how parallel programs scale, we need a working mental model of memory and the cost of touching it.

The address space abstraction

> **Definition** [address space]. A computer's memory is organised as an array of bytes. Each byte is identified by an integer address — its position in this array. A program's address space is the set of addresses it can read and write. We assume memory is byte-addressable, so the smallest unit accessible at a given address is one byte.

A processor interacts with this array through two new instructions. A load reads a value from memory into a register; a store writes the contents of a register back to memory. Symbolically,

```
ld R0 <- mem[R2]    ; load 4 bytes at the address held in R2 into R0
st mem[R2] <- R0    ; store R0 to the address held in R2
```

Everything a program "computes" is, from the processor's perspective, a sequence of such loads, stores, and arithmetic operations on registers.

Latency and stalls

> **Definition** [memory access latency]. The latency of a memory access is the amount of time, measured in clock cycles or nanoseconds, between when the processor requests data and when that data is delivered.

A processor stalls when it cannot make progress because the next instruction depends on a previous instruction that has not yet completed. Memory accesses are a major source of stalls because their latency is large compared to the time to execute an arithmetic instruction. For the sequence

```
ld  r0 mem[r2]
ld  r1 mem[r3]
add r0, r0, r1
```

the `add` cannot issue until both loads have returned, and a load to main memory typically costs hundreds of cycles. The processor sits idle the whole time.

> **Why this matters for parallel code.** *A program that is otherwise embarrassingly parallel will fail to scale if its threads spend most of their cycles waiting for memory. Every technique we study later — multi-threading, prefetching, locality optimisation, message passing — exists to attack some flavour of this problem.*

Caches

The standard hardware response to high memory latency is the CACHE.

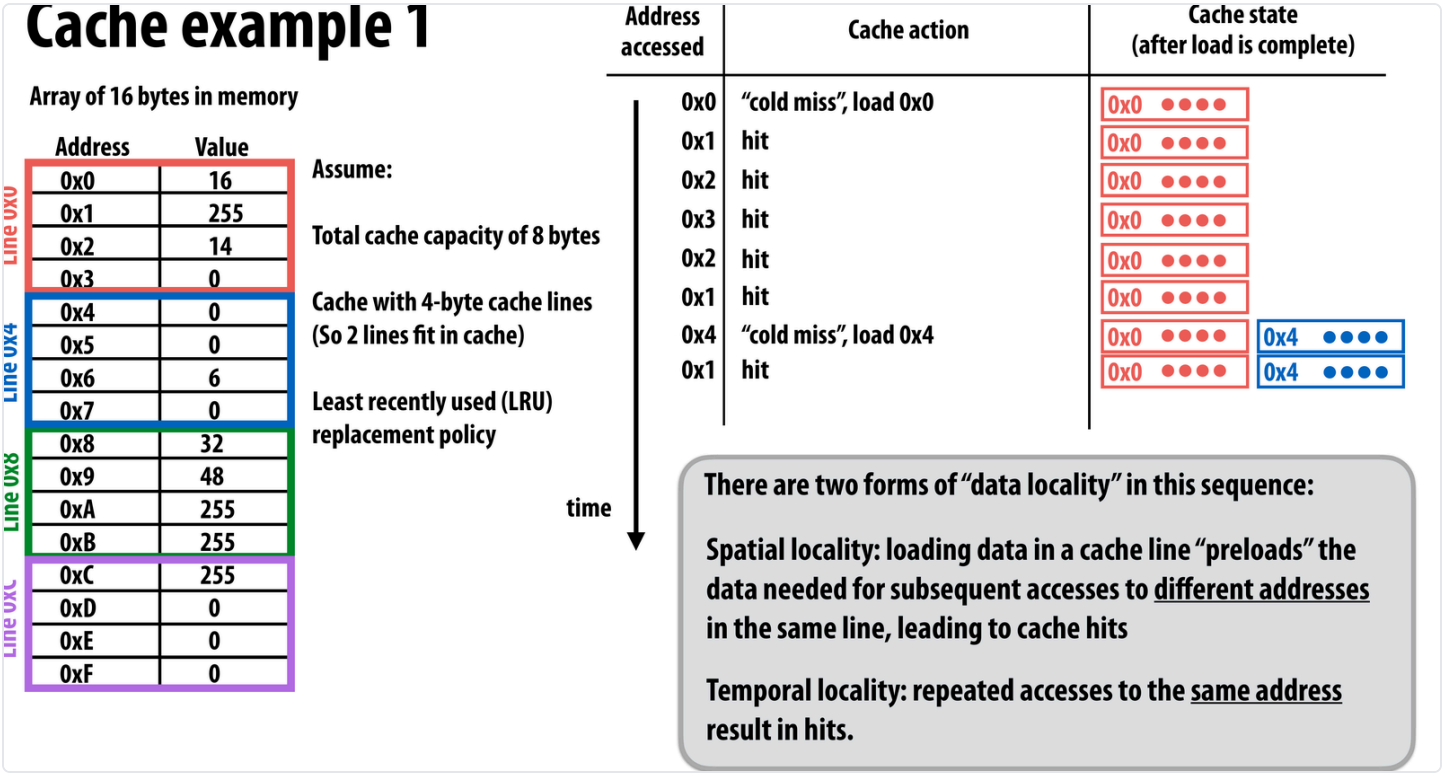
> **DEFINITION** [CACHE]. A CACHE is on-chip storage that holds a copy of a subset of memory. If the address the processor needs is in the cache, the data can be delivered in a few cycles rather than the hundreds required to access DRAM. A cache is purely a performance feature: it does not change what a program computes, only how quickly it computes it.

Caches transfer data at the granularity of fixed-size CACHE LINES (typically 64 bytes on x86), not individual bytes. The unit of bookkeeping is the line, and pulling one byte into the cache pulls its entire enclosing line. This single design choice has a large payoff because real programs exhibit two forms of LOCALITY that the line granularity exploits.

> **DEFINITION** [TEMPORAL AND SPATIAL LOCALITY]. A reference stream has TEMPORAL LOCALITY if recently accessed addresses are likely to be accessed again soon, and SPATIAL LOCALITY if addresses near a recently accessed address are likely to be accessed soon.

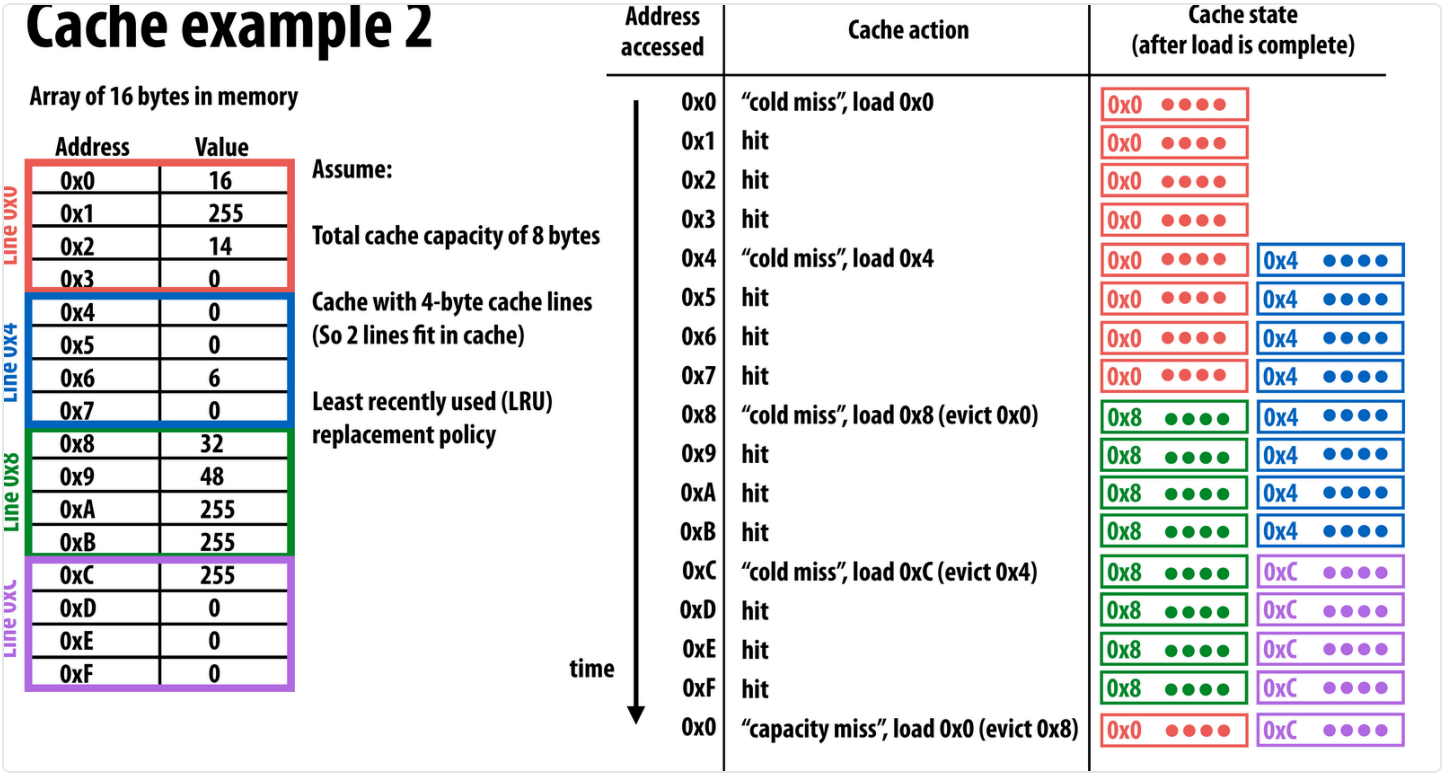
Spatial locality makes line-granularity transfer cheap on a per-byte basis: the cost of fetching one byte already paid for the next 63. Temporal locality makes the cache profitable to maintain at all: re-reads of a hot location stay on-chip.

> **EXAMPLE** [SEQUENTIAL SCAN OF AN ARRAY]. Suppose the cache holds two lines of 4 bytes each with an LRU replacement policy, and a program reads addresses $0\text{x}0, 0\text{x}1, 0\text{x}2, \ldots, 0\text{x}F$ in order. Address $0\text{x}0$ is a COLD MISS and pulls the line $[0\text{x}0, 0\text{x}3]$ into the cache; the next three accesses are hits (spatial locality). At $0\text{x}4$ another cold miss pulls $[0\text{x}4, 0\text{x}7]$, and so on — each 4-byte line costs one miss and three hits.



Sequential scan: each 4-byte line gives one miss and three hits, hit rate $\frac{12}{16} = 75\%$.

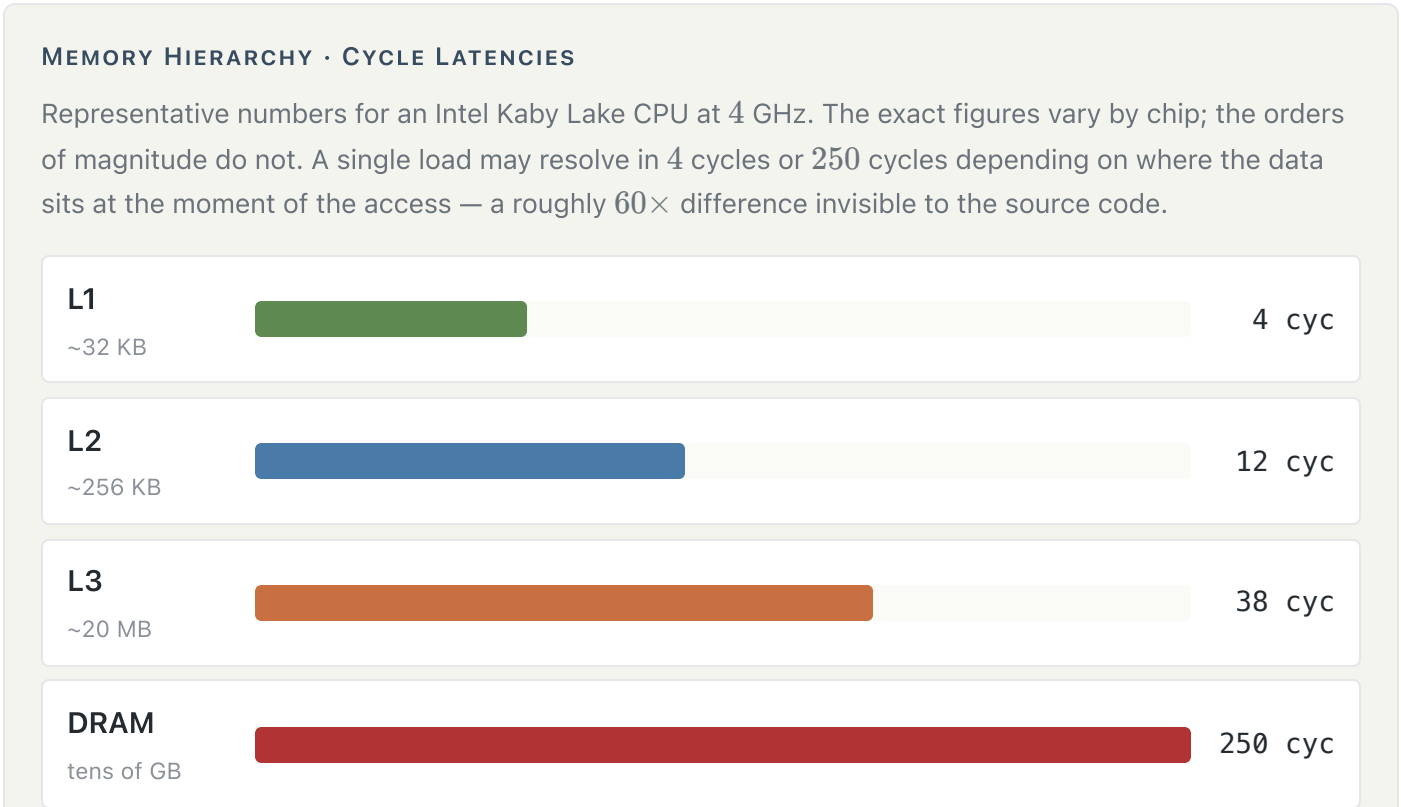
Example [capacity miss]. With the same 2-line cache, suppose the program reads the entire 16-byte array and then re-reads address 0x0. The four scan misses have evicted the original line: 0x0's line was kicked out when 0x8's line was loaded. The re-read of 0x0 is therefore a **capacity miss** — the data was once in the cache but did not fit alongside the rest of the working set. Cold misses are unavoidable for the first touch; capacity misses are programmer-addressable through better blocking.



THE SAME WORKLOAD PLUS A RE-READ OF 0x0: THE ORIGINAL LINE HAS BEEN EVICTED, SO IT RETURNS AS A CAPACITY MISS.

The memory hierarchy

Caches are not a single tier. A modern CPU exposes a hierarchy of caches of increasing size and increasing latency, backed by DRAM.



Bar width is logarithmic in latency.

1.6 Summary

Single-thread-of-control performance is improving very slowly. To run programs significantly faster, those programs must use multiple processing elements or specialised hardware — which means the programmer needs to reason about parallel execution rather than rely on the next generation of silicon. Writing parallel programs is non-trivial: it requires decomposing the work, balancing the load, and managing communication and synchronisation, and doing all of that well requires knowing what the machine looks like and, in particular, how data moves through it. The payoff is large: modern hardware has more processing capability than naïve sequential code reveals, and the rest of the course is about how to actually use it.